SDG BLOCKCHAIN ACCELERATOR

Mentor Feedback Report: SolarVillage (CUBID)

Program: EMURGO : UNDP Blockchain Accelerator (Cohort 2)

Project: SolarVillage (CUBID Protocol)

Focus Area: Rural Energy Infrastructure, Carbon Accounting, Mini-grid Settlement Systems

Mentorship Focus: Technical Architecture, PoC Execution Strategy, Data Integrity & System Design

Total Mentoring Sessions: 4

Mentor(s): Paul Kruegel / Cong Lee

1. Where the Project Stood

SolarVillage came into the accelerator in a very different place compared to most teams. While others were still figuring out what to build, this team had already designed a fairly complete system , not just a concept, but a full architecture spanning energy settlement, payments, and carbon issuance.

What stood out immediately is that the team wasn't trying to "add blockchain" to a problem. They were thinking in terms of infrastructure , how energy data is captured, how it becomes financially meaningful, and how it can eventually support something like carbon credits in a credible way.

Because of that, the mentoring conversations were less about *what* they're building, and more about *how to actually make it real without getting stuck in their own complexity*.

2. Shifting the Conversation Toward the PoC

A big part of the mentoring process was reframing the way the team thinks about the pilot.

The architecture, as designed, is quite extensive. It includes Hydra for settlement, multiple smart contract components, token flows for energy accounting, and a full pathway to carbon issuance. Technically, it all makes sense. But for a pilot, trying to implement everything at once is where things can break down.

So the focus became: what actually needs to be proven first?

Not carbon markets.
Not full token design.
Not even perfect on-chain execution.

The real question is whether the system can reliably move from:
meter → data → settlement → auditable record

If that works, everything else can follow. If it doesn't, the rest doesn't matter.

That shift , from building the full system to validating the core loop , was probably the most important outcome of the mentoring.

3. Thoughts on the Architecture (Through a PoC Lens)

The architecture itself is strong. The separation between Hydra as the operational layer and Cardano L1 as the audit and issuance layer is well thought out and fits the use case. It shows a clear understanding of where high-frequency activity should live versus where long-term guarantees are needed.

That said, Hydra introduces real operational overhead. Running it per microgrid, coordinating it with off-chain services, and ensuring stability in a rural environment is not trivial. None of that makes it the wrong choice , it just means it shouldn't become the bottleneck for the pilot.

A similar observation came up around the carbon layer. The WH-to-COG logic is one of the most interesting parts of the system, and it's clearly designed with MRV and auditability in mind. But at the PoC stage, carbon issuance isn't the hard part. The hard part is making sure the underlying data is trustworthy.

In a way, the project is already solving the "future" problem very well. The risk is underestimating the difficulty of the "present" one , getting clean, consistent, reliable data from real-world systems.

4. Where the Real Complexity Actually Is

One thing that became clearer over time is that most of the technical risk doesn't sit in the smart contracts.

It sits in everything around them.

Metering, data ingestion, tariff logic, payment integrations, backend coordination , these are the parts that will either make or break the pilot. The blockchain layer can be perfectly designed, but if the inputs are inconsistent or delayed, the system loses credibility.

So a lot of the mentoring discussions ended up focusing on things that aren't usually highlighted in blockchain projects:

- how meter data is validated
- how often it's aggregated
- how failures are handled
- how operators interact with the system day-to-day

In other words, the system has to work operationally before it works cryptographically.

5. Simplifying Without Losing the Vision

Another important theme was simplification.

There's a natural tendency, especially with a strong technical team, to build the full version from the start. In this case, that would mean:

- all validators in place
- full token lifecycle
- complete Hydra setup
- carbon issuance logic running end-to-end

The concern is that this creates too many moving parts too early.

The recommendation wasn't to change the architecture, but to **sequence it**. Start with a version that proves the fundamentals, even if parts of it are simulated or simplified. Then layer in complexity once those fundamentals are stable.

That way, progress is visible and measurable, instead of blocked by integration issues.

6. What the Team Is Doing Well

One of the strongest aspects of SolarVillage is that it's clearly grounded in reality. The decision to use custodial models, support USSD, and account for low-connectivity environments shows that the team understands the users and the context.

There's also a level of technical clarity that's rare at this stage. The way energy is tracked, transformed, and eventually linked to carbon logic is not hand-wavy , it's defined, structured, and auditable.

And importantly, the team seems open to adjusting how they get there, even if the end vision remains ambitious.

7. Where to Be Careful

The main risk isn't that the system won't work. It's that it tries to do too much too early.

There are a lot of dependencies:

- hardware
- backend services
- blockchain infrastructure
- external partners

If too many of these are required to be “perfect” at the same time, the pilot can stall.

There’s also a subtle risk around focus. The carbon narrative is compelling, but it can pull attention away from the core job of the system , which is to produce reliable, verifiable operational data.

If that foundation is strong, carbon becomes a natural extension. If not, it becomes premature.

8. Closing Thoughts

SolarVillage is one of the few projects in the cohort that already feels like it could become real infrastructure, not just a demo.

The architecture is not the issue. If anything, it’s ahead of where it needs to be for a pilot.

The real challenge now is execution discipline , being able to step back from the full vision, focus on the smallest version that proves the system works, and build from there.

If the team manages that balance, they’re in a very strong position to move from a well-designed system to something that actually runs in the field and scales beyond the pilot.